

Cyberster Internship Program

Internship Report

TRACK: BLUE TEAM

NAME: MUHAMMAD RAZA

ROLL NUMBER: CSI-B1-121

WEEK 2: DETECTION, CUSTOM RULES, AND LOG ANALYSIS

TABLE OF CONTENTS

Overview 5

Days 8 and 9 — File Integrity Monitoring (FIM) 5

Linux FIM Configuration and Testing (Day 8) 5

Windows 10 FIM Configuration and Testing (Day 9) 7

Days 10 and 11 — Custom Detection Rules 10

Rule 1: New User Creation 10

Rule 2: SSH Brute Force 11

Rule 3: Unauthorized USB Insertion 13

Days 12 and 13 — Log Analysis Fundamentals 14

Log Analysis Breakdown 14

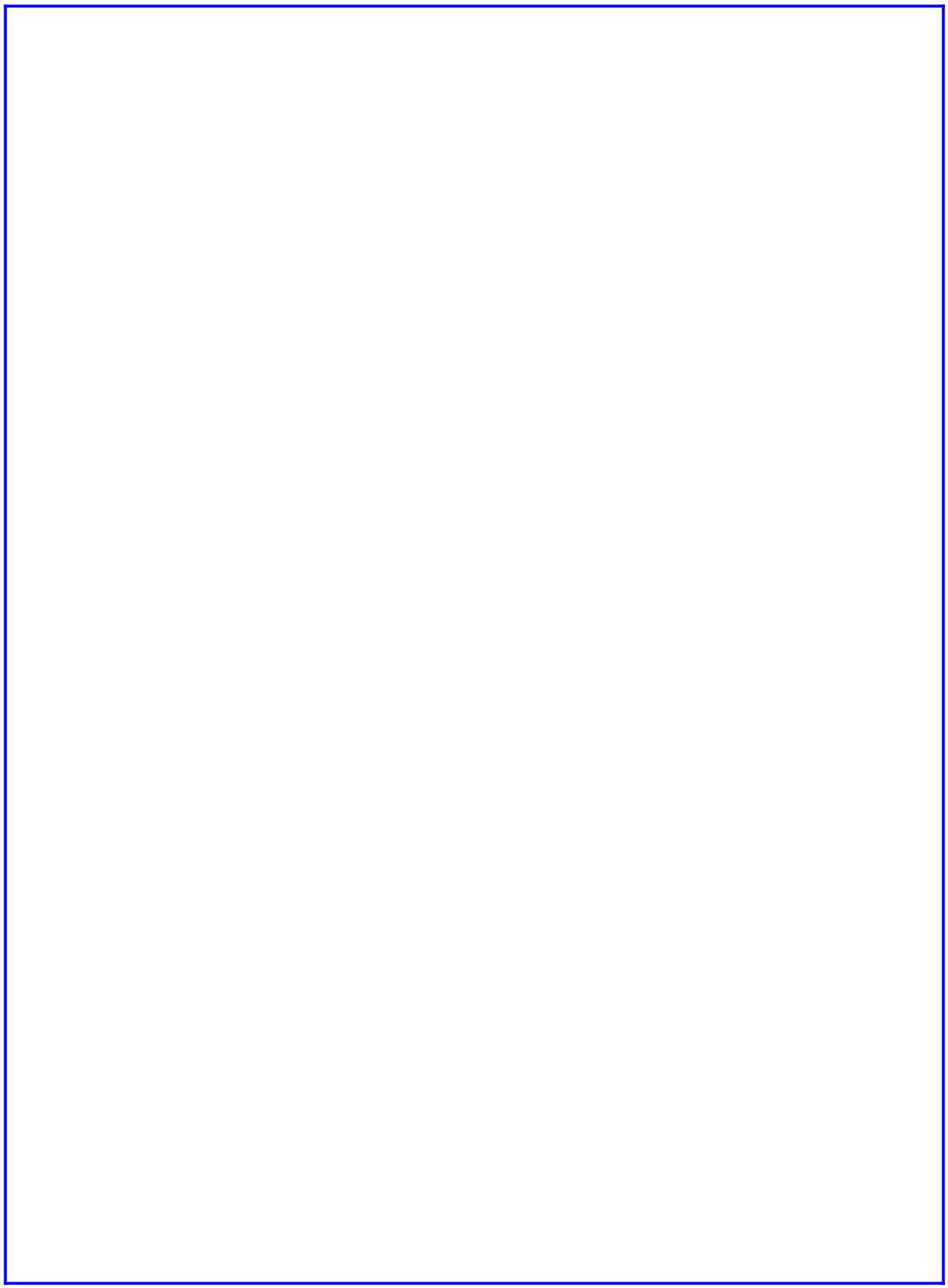
Timeline of Suspicious Activity 18

Day 14 — Wazuh Active Response 19

Configuration Workflow 20

Attack and Verification 21

Risks and Safeguards of Active Response 22



Overview

Week 2 transitions the SOC environment from a passive logging infrastructure to an active detection and response platform. The objective of this week was to configure real-time File Integrity Monitoring (FIM), engineer custom detection rules to catch specific MITRE ATT&CK techniques, perform deep-dive log analysis across Linux and Windows environments, and finally, implement automated Active Response to dynamically block attackers at the OS firewall level.

Days 8 and 9 — File Integrity Monitoring (FIM)

File Integrity Monitoring (FIM) is critical for detecting unauthorized modifications to system files, configuration files, and critical directories.

Linux FIM Configuration and Testing (Day 8)

To monitor the critical /etc directory on the Kali Linux agent, I modified the local ossec.conf file.

Step-by-Step Execution:

1. Opened the agent configuration file: sudo nano /var/ossec/etc/ossec.conf
2. Added the directory to the <syscheck> block with real-time reporting enabled:
   <directories realtime="yes" report_changes="yes">/etc</directories>
3. Set the syscheck frequency to 300 seconds.
4. Restarted the Wazuh agent to apply changes: sudo systemctl restart wazuh-agent

```
<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>300</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Directories to check  (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>
  <directories realtime="yes" report_changes="yes">/etc</directories>
  <directories realtime="yes">/var/log</directories>
```

The Attack & Verification:

To test the configuration, I simulated an attacker establishing persistence by creating, modifying, and deleting a file in the /etc directory.

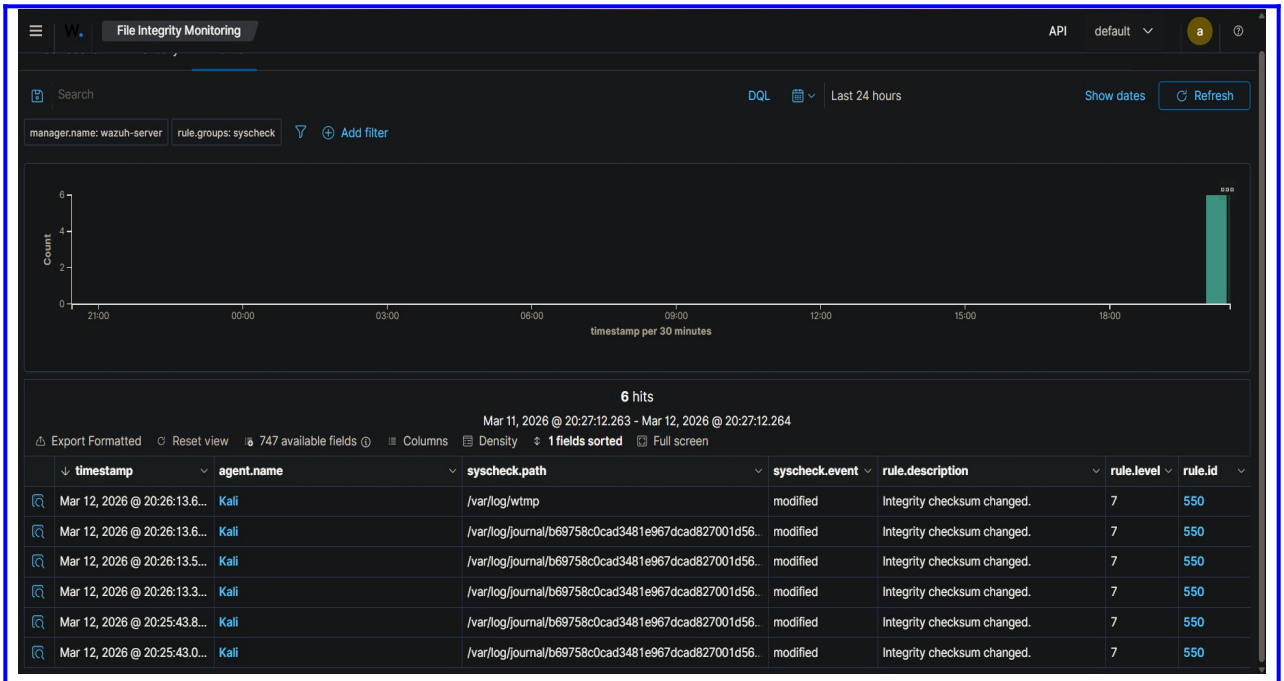
- Command 1: sudo touch /etc/hacker_test.txt
- Command 2: echo "Attacker persistence established" | sudo tee /etc/hacker_test.txt
- Command 3: sudo rm /etc/hacker_test.txt

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo touch /etc/hacker_test.txt
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ echo "Attacker persistence established" | sudo tee /etc/hacker_test.txt
Attacker persistence established

┌──(kali㉿kali)-[~]
└─$ sudo rm /etc/hacker_test.txt
```

The Wazuh Dashboard immediately caught the modifications, triggering Level 7 "Integrity checksum changed" alerts for the Kali agent.



Windows 10 FIM Configuration and Testing (Day 9)

I repeated this process for the Windows 10 endpoint, specifically targeting the C:\Windows\System32 directory where malware often drops malicious payloads.

Step-by-Step Execution:

1. Opened C:\Program Files (x86)\ossec-agent\ossec.conf using Notepad as Administrator.
2. Configured the <syscheck> block:

   <directories realtime="yes"

   report_changes="yes">C:\Windows\System32</directories>

```
<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>300</frequency>

<!-- Default files to be monitored. -->
<directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$">%WINDIR%</directories>

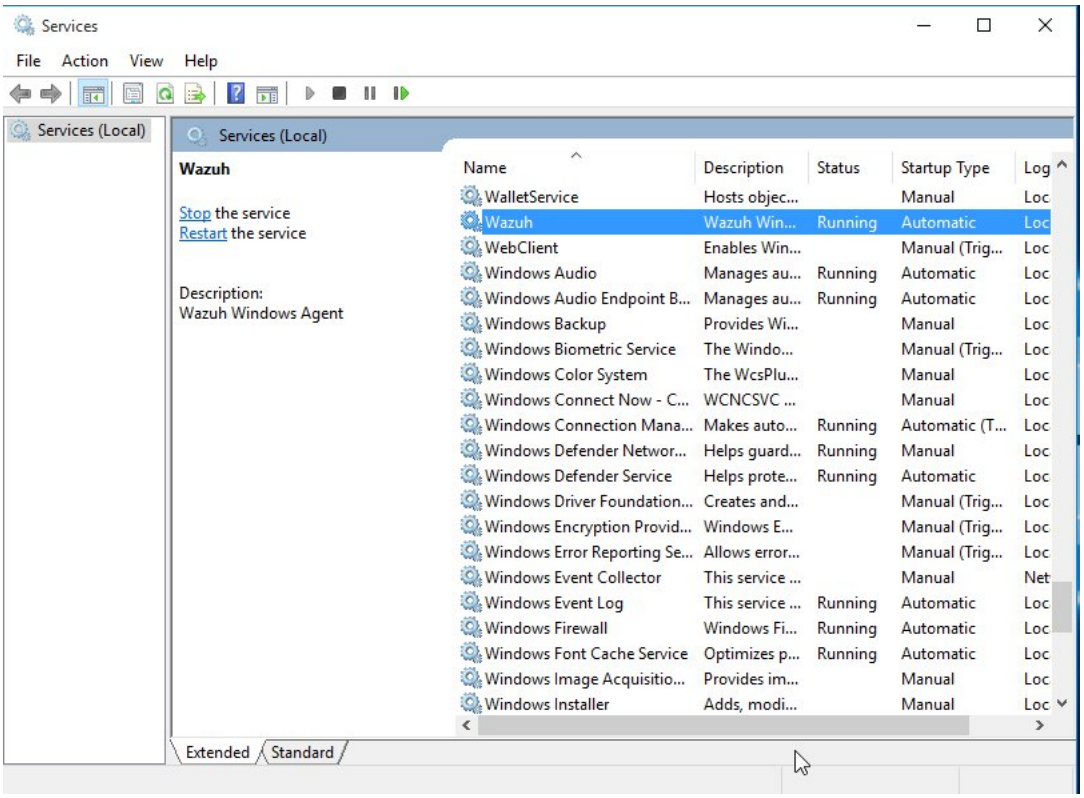
<directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|
<directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
<directories recursion_level="0" restrict="WMIC.exe$">%WINDIR%\SysNative\wbem</directories>
<directories recursion_level="0" restrict="powershell.exe$">%WINDIR%\SysNative\WindowsPowerShell\v1.
<directories recursion_level="0" restrict="winrm.vbs$">%WINDIR%\SysNative</directories>

<!-- 32-bit programs. -->
<directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|
<directories recursion_level="0">%WINDIR%\System32\drivers\etc</directories>
<directories recursion_level="0" restrict="WMIC.exe$">%WINDIR%\System32\wbem</directories>
<directories recursion_level="0" restrict="powershell.exe$">%WINDIR%\System32\WindowsPowerShell\v1.0
<directories recursion_level="0" restrict="winrm.vbs$">%WINDIR%\System32</directories>

<directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup</directories
<directories realtime="yes" report_changes="yes">C:\Windows\System32</directories>

<ignore>%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup\desktop.ini</ignore>
```

3. Restarted the Wazuh Windows Agent via services.msc.



The Attack & Verification:

Using the Windows Command Prompt, I simulated a malware payload drop:

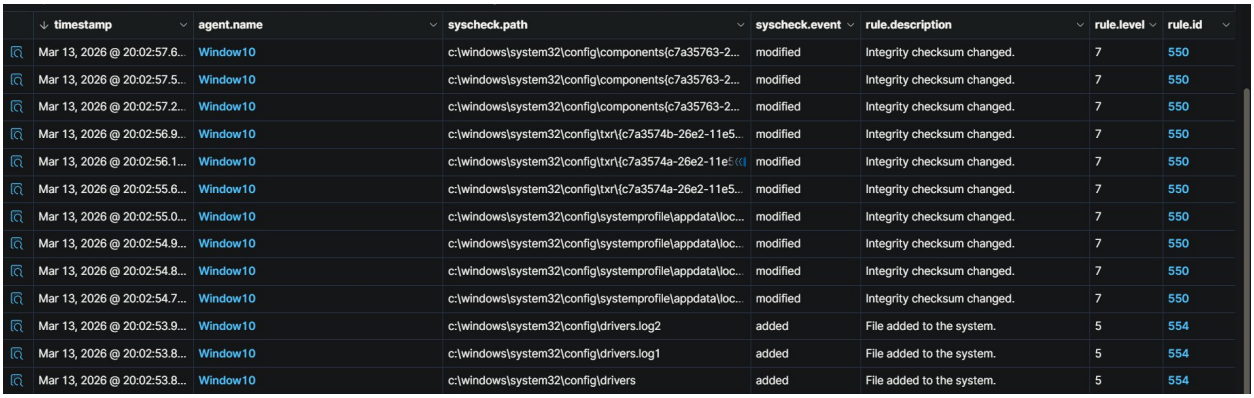
- echo "Malicious payload dropped" > C:\Windows\System32\hacker_test.txt
- echo "Updating payload" >> C:\Windows\System32\hacker_test.txt
- del C:\Windows\System32\hacker_test.txt

```
C:\Windows\system32>echo "Malicious payload dropped" > C:\Windows\System32\hacker_test.txt

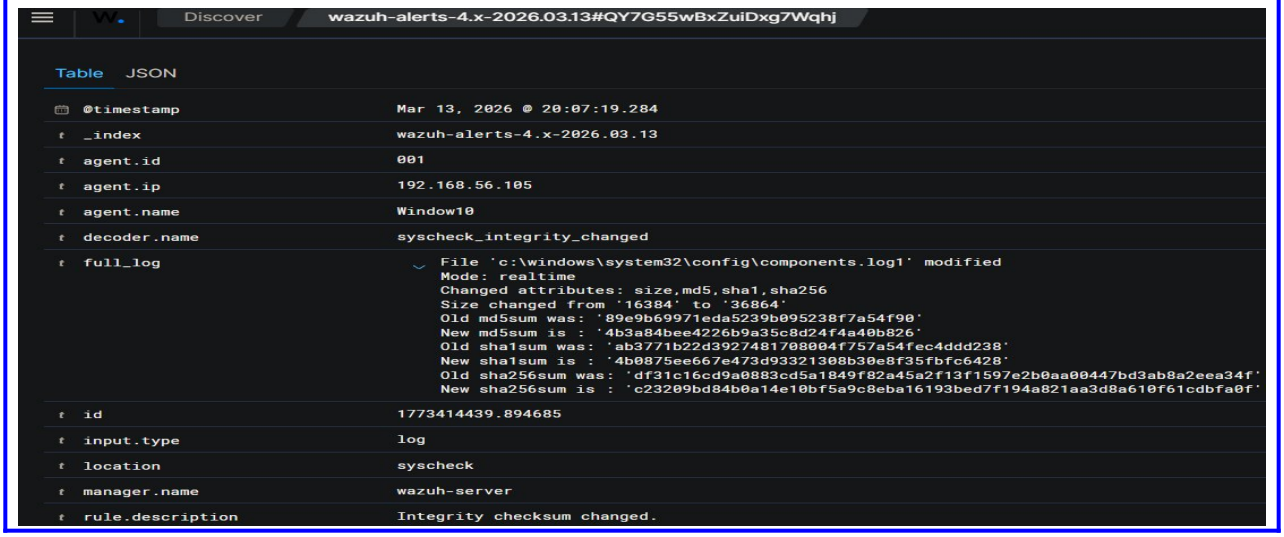
C:\Windows\system32>echo "Updating payload" >> C:\Windows\System32\hacker_test.txt

C:\Windows\system32>del C:\Windows\System32\hacker_test.txt
```

Wazuh successfully captured these events. Expanding the JSON data revealed the exact SHA256, MD5, and SHA1 hash changes of the modified files.



timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Mar 13, 2026 @ 20:02:57.6...	Window10	c:\windows\system32\config\components{c7a35763-2...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:57.5...	Window10	c:\windows\system32\config\components{c7a35763-2...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:57.2...	Window10	c:\windows\system32\config\components{c7a35763-2...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:56.9...	Window10	c:\windows\system32\config\txr\{c7a3574b-26e2-11e5...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:56.1...	Window10	c:\windows\system32\config\txr\{c7a3574a-26e2-11e5...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:55.6...	Window10	c:\windows\system32\config\txr\{c7a3574a-26e2-11e5...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:55.0...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:54.9...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:54.8...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:54.7...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:53.9...	Window10	c:\windows\system32\config\drivers.log2	added	File added to the system.	5	554
Mar 13, 2026 @ 20:02:53.8...	Window10	c:\windows\system32\config\drivers.log1	added	File added to the system.	5	554
Mar 13, 2026 @ 20:02:53.8...	Window10	c:\windows\system32\config\drivers	added	File added to the system.	5	554



Days 10 and 11 — Custom Detection Rules

Default SIEM rules are often noisy or insufficient. These tasks required engineering custom XML rules to detect specific attacker behaviors mapped to the MITRE ATT&CK framework.

Rule 1: New User Creation

To detect unauthorized persistence, I wrote a custom rule (ID 100010) to trigger a Level 10 Critical alert whenever a new user account is created on a Linux system (T1136).

```
[wazuh-user@wazuh-server ~]$ sudo nano /var/ossec/etc/rules/local_rules.xml
```

```
<!-- Custom Rule 1: New User Creation -->
<rule id="100010" level="10">
 <if_sid>5902</if_sid>
 <description>CRITICAL: New user account created on the system</description>
 <mitre>
  <id>T1136</id>
 </mitre>
</rule>
```

```
[wazuh-user@wazuh-server ~]$ sudo systemctl restart wazuh-manager
```

Verification: I executed sudo useradd darthvader on Kali. The rule triggered perfectly.

```
┌──(kali㉿kali)-[~]
└─$ sudo useradd darthvader
sudo] password for kali:
```

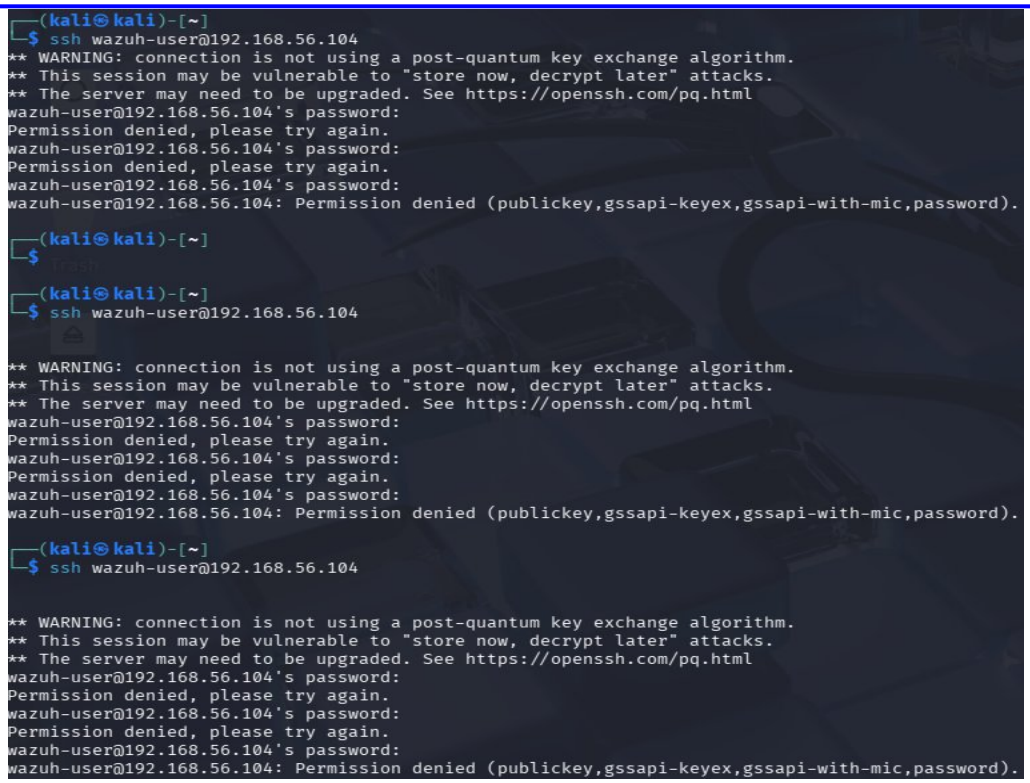
t	agent.ip	192.168.56.107
t	agent.name	Kali
t	data.dstuser	darthvader
t	data.gid	1001
t	data.home	/home/darthvader
t	data.shell	/bin/sh,
t	data.uid	1001
t	decoder.name	useradd
t	decoder.parent	useradd
t	full_log	Mar 14 18:34:13 kali useradd[17605]: new user: name=darthvader, UID=1001, GID=1001, home=/home/darthvader, shell=/bin/sh, from=/dev/pts/1
t	id	1773513253.592111
t	input.type	log
t	location	journald
t	manager.name	wazuh-server
t	predecoder.hostname	kali
t	predecoder.program_name	useradd
t	predecoder.timestamp	Mar 14 18:34:13
t	rule.description	CRITICAL: New user account created on the system
#	rule.firedtimes	1
t	rule.groups	local, syslog, sshd
t	rule.id	100010
#	rule.level	10

Rule 2: SSH Brute Force

To detect credential stuffing, I created a frequency-based rule (ID 100011). It triggers a Level 12 alert if 6 authentication failures (<if_matched_sid>5760</if_matched_sid>) occur from the exact same source IP within a 120-second timeframe (T1110.001).

```
<!-- Custom Rule 2: SSH Brute Force -->
<rule id="100011" level="12" frequency="6" timeframe="120">
 <if_matched_sid>5760</if_matched_sid>
 <same_source_ip />
 <description>CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from same IP)</description>
 <mitre>
  <id>T1110.001</id>
 </mitre>
</rule>
```

Verification: I spammed the SSH login prompt with incorrect passwords. Wazuh grouped the events and fired the Level 12 Critical alert.



```
┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
wazuh-user@192.168.56.104: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

┌──(kali㉿kali)-[~]
└─$

┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104

** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
wazuh-user@192.168.56.104: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104

** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
wazuh-user@192.168.56.104: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
```

Discover | wazuh-alerts-4.x-2026.03.14#rsLI7ZwBo0CZirNxEch_

t data.dstuser	wazuh-user
t data.srcip	192.168.56.107
t data.srcport	57490
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Mar 14 19:48:53 wazuh-server sshd[5331]: Failed password for wazuh-user from 192.168.56.107 port 57490 ssh2
t id	1773517735.963663
t input.type	log
t location	journald
t manager.name	wazuh-server
t predecoder.hostname	wazuh-server
t predecoder.program_name	sshd
t predecoder.timestamp	Mar 14 19:48:53
t previous_output	Mar 14 19:48:52 wazuh-server sshd[5329]: Failed password for wazuh-user from 192.168.56.107 port 57488 ssh2 Mar 14 19:48:52 wazuh-server sshd[5329]: Failed password for wazuh-user from 192.168.56.107 port 57488 ssh2 Mar 14 19:48:50 wazuh-server sshd[5327]: Failed password for wazuh-user from 192.168.56.107 port 57476 ssh2 Mar 14 19:48:50 wazuh-server sshd[5327]: Failed password for wazuh-user from 192.168.56.107 port 57476 ssh2 Mar 14 19:48:49 wazuh-server sshd[5325]: Failed password for wazuh-user from 192.168.56.107 port 57466 ssh2
t rule.description	CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from same IP)
# rule.firedtimes	2
# rule.frequency	6
t rule.groups	local, syslog, sshd
t rule.id	100011
# rule.level	12

Rule 3: Unauthorized USB Insertion

To detect physical data exfiltration (T1052), I created Rule 100012 to upgrade standard USB mounting logs into a Level 8 Warning.

```
<!-- Custom Rule 3: Unauthorized USB plugged in -->
<rule id="100012" level="8">
 <if_sid>81100</if_sid>
 <description>WARNING: Unauthorized USB Storage Device Plugged In</description>
 <mitre>
  <id>T1052</id>
 </mitre>
</rule>
```

Verification: Due to agent-side OS logging limitations, I utilized the backend wazuh-logtest engine to prove the rule logic. Injecting a raw kernel USB log successfully fired Phase 3 of the testing engine, confirming the rule's operational status.

```
[wazuh-user@wazuh-server ~]$ echo "Mar 15 12:00:00 kali kernel: [  123.456789] usb 1-1: USB Mass Storage device detected
" | sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.3
Type one log per line


**Phase 1: Completed pre-decoding.
       full event: 'Mar 15 12:00:00 kali kernel: [  123.456789] usb 1-1: USB Mass Storage device detected'
       timestamp: 'Mar 15 12:00:00'
       hostname: 'kali'
       program_name: 'kernel'

**Phase 2: Completed decoding.
       name: 'kernel'
       parent: 'kernel'
       id: 'usb'

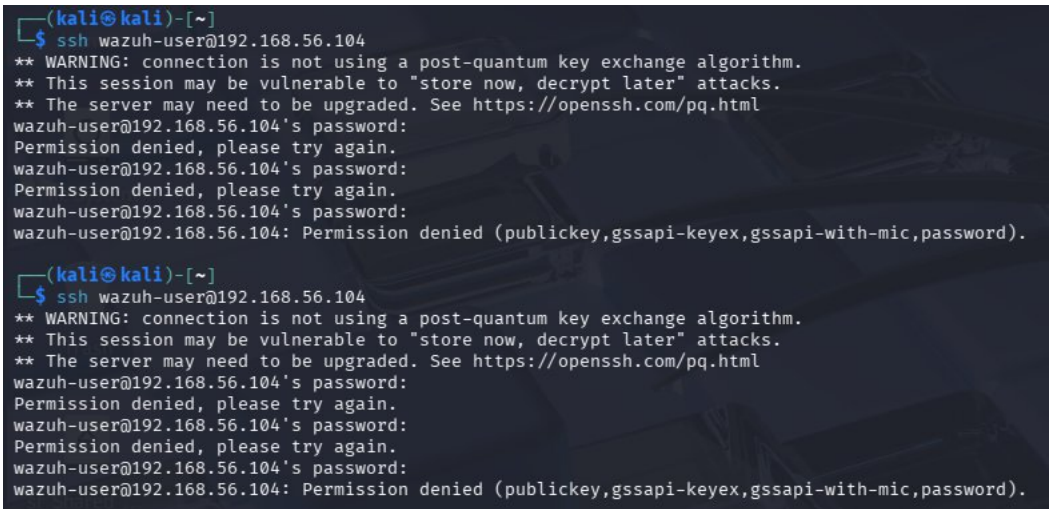
**Phase 3: Completed filtering (rules).
       id: '100012'
       level: '8'
       description: 'WARNING: Unauthorized USB Storage Device Plugged In'
       groups: '['local', 'syslog', 'sshd']'
       firedtimes: '1'
       mail: 'False'
       mitre.id: '['T1052']'
       mitre.tactic: '['Exfiltration']'
       mitre.technique: '['Exfiltration Over Physical Medium']'
```

Days 12 and 13 — Log Analysis Fundamentals

Log analysis is the core of incident response. This phase involved querying the SIEM to correlate events across multiple agents and identify the "who, what, and when" of a security incident.

Log Analysis Breakdown

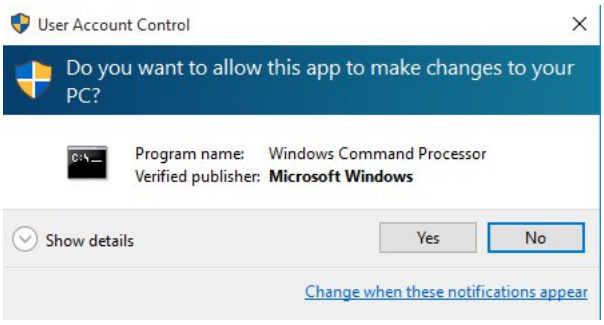
Source OS: Linux
Event / Rule ID: Wazuh Rule 5760
Trigger Activity
Multiple sshd: authentication failed events for wazuh-user from 192.168.56.107.
Analyst Interpretation
Deliberate Attack: The frequency and volume of these failures indicate a targeted SSH Brute Force attack, not a routine typo.
Screenshot Evidence

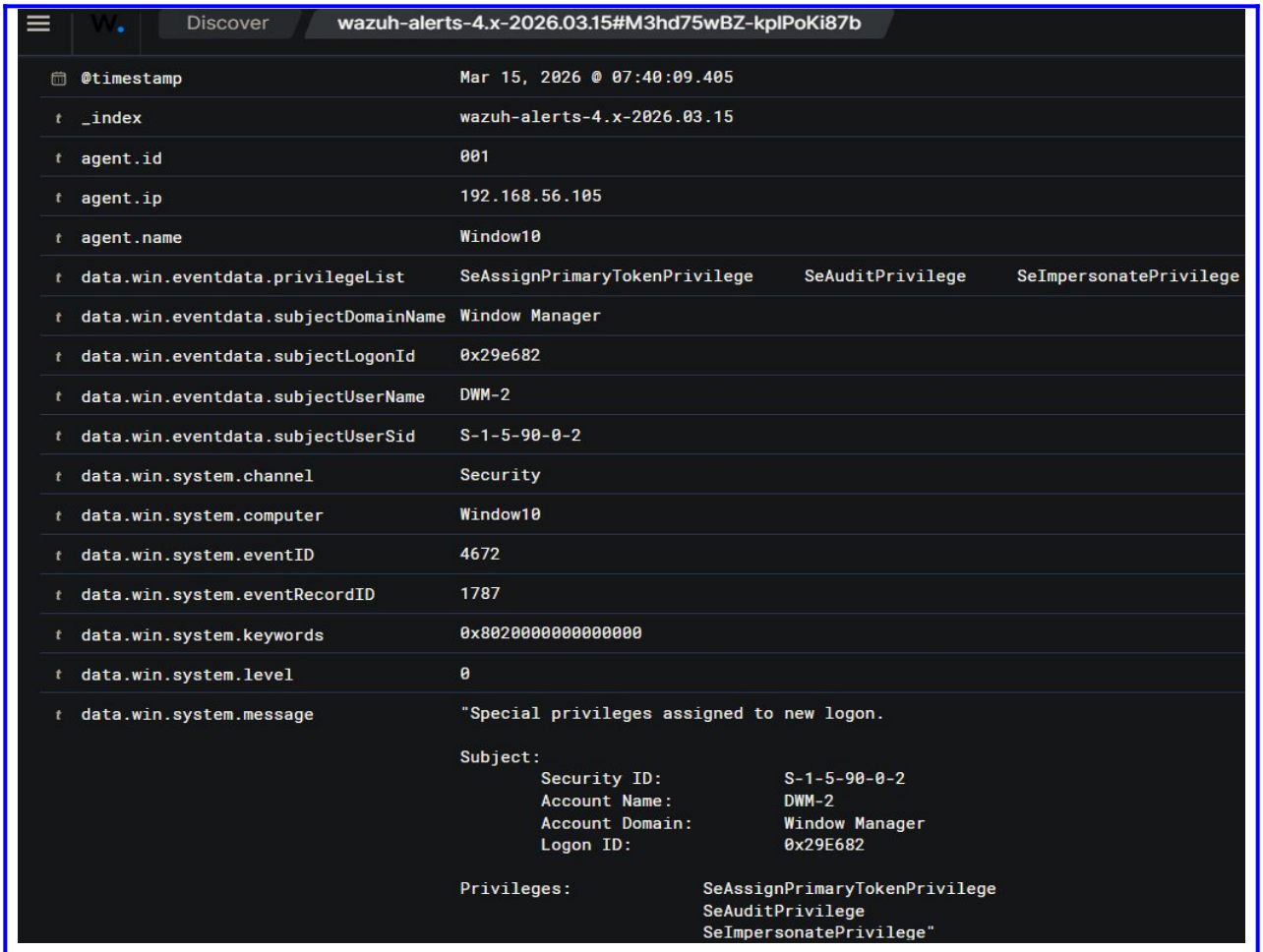


Discover | wazuh-alerts-4.x-2026.03.15#nXg875wBZ-kplPoK7s0O

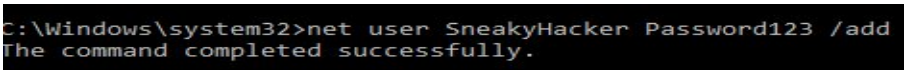
t full_log	Mar 15 02:03:38 wazuh-server sshd[3990]: Failed password for wazuh-user from 192.168.56.1 port 58269 ssh2
t id	1773540219.9424
t input.type	log
t location	journald
t manager.name	wazuh-server
t predecoder.hostname	wazuh-server
t predecoder.program_name	sshd
t predecoder.timestamp	Mar 15 02:03:38
t rule.description	sshd: authentication failed.
# rule.firedtimes	1
t rule.gdpr	IV_35.7.d, IV_32.2
t rule.gpg13	7.1
t rule.groups	syslog, sshd, authentication_failed
t rule.hipaa	164.312.b
t rule.id	5760
# rule.level	5
rule.mail	false
t rule.mitre.id	T1110.001, T1021.004
t rule.mitre.tactic	Credential Access, Lateral Movement
t rule.mitre.technique	Password Guessing, SSH

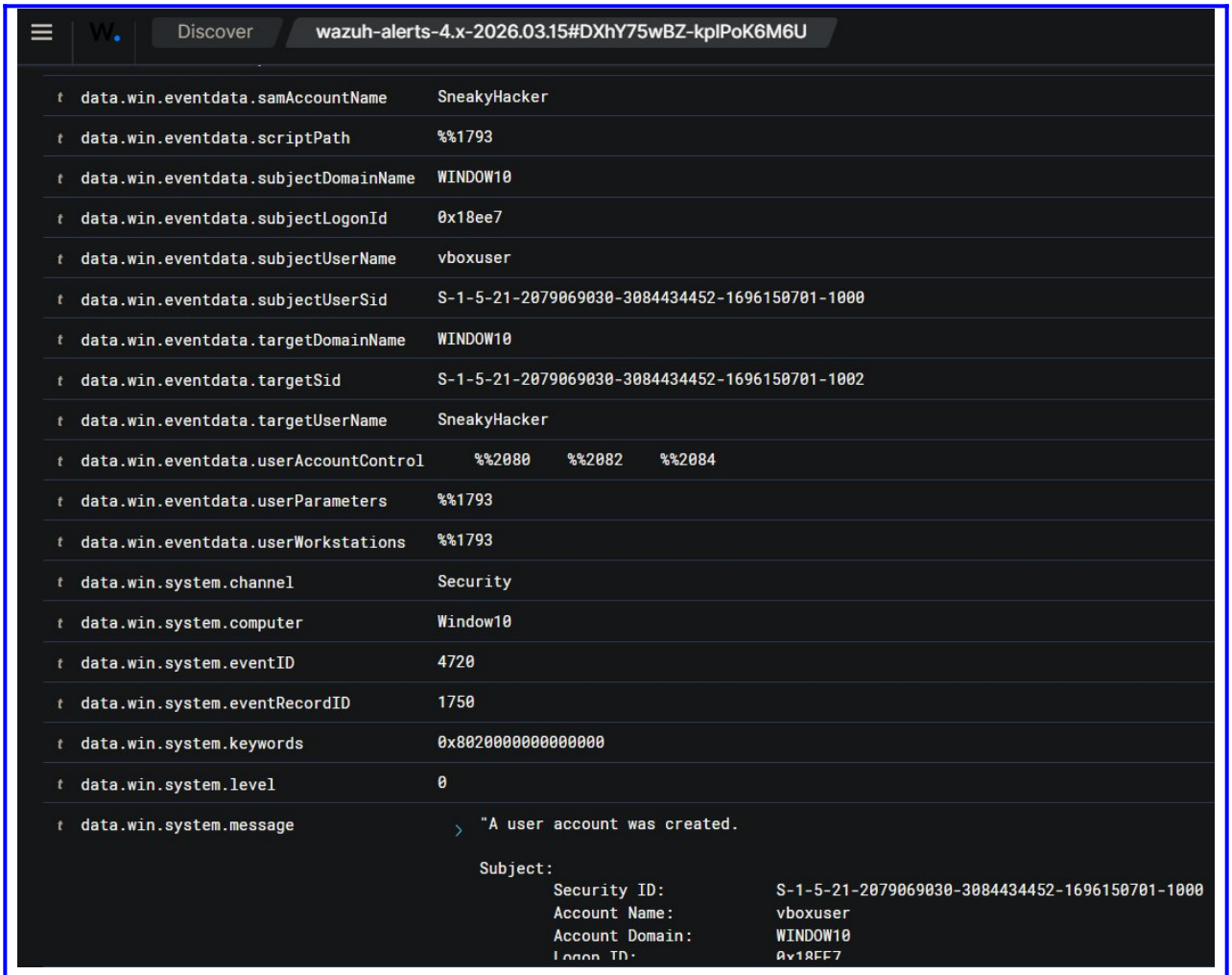
Source OS: Windows
Event / Rule ID: Event ID 4672
Trigger Activity
Elevated Command Prompt assigning special privileges to a logon session.
Analyst Interpretation
Suspicious if Unplanned: Routine if done by a known Admin, but highly suspicious if correlated with a prior brute-force alert.
Screenshot Evidence





Source OS: Windows
Event / Rule ID: Event ID 4720
Trigger Activity
net user SneakyHacker command executed, creating a new account.
Analyst Interpretation
Critical: Active establishment of persistence. The logs identify vboxuser as the creator of SneakyHacker.
Screenshot Evidence





Source OS: Windows
Event / Rule ID: Event ID 4625
Trigger Activity
vboxuser failed logon attempt with SubStatus 0xc000006a
Analyst Interpretation
Routine (Isolated): The specific sub-status code 0xc000006a indicates a bad password was entered.
Screenshot Evidence



Timeline of Suspicious Activity

The following timeline represents the chronological flow of flagged incidents across the environment.

Timestamp	Event Description	Agent	Analyst Note
Mar 14, 18:34	CRITICAL: New user account created (Level 10)	Kali	Flagged as potential backdoor creation (User: darthvader).
Mar 14, 19:48	CRITICAL: SSH Brute Force Attempt (Level 12)	Wazuh-Server	Active credential stuffing attack identified from 192.168.56.107.
Mar 15, 07:34	User account enabled or created (Level 8)	Window10	Windows Event ID 4720 triggered; persistence suspected.
Mar 15, 07:40	Special privileges assigned to new logon (Level 3)	Window10	Windows Event ID 4672 triggered; potential privilege escalation.

Day 14 — Wazuh Active Response

Active response elevates the SIEM from a passive monitor to an automated defense mechanism. I configured Wazuh to automatically drop traffic from an IP address at the OS firewall level if it triggered the SSH Brute Force rule.

Configuration Workflow

1. Adjusted Custom Rule 100011 to trigger exactly on 5 failed attempts (frequency="5").

```
<!-- Custom Rule 2: SSH Brute Force -->
<rule id="100011" level="12" frequency="5" timeframe="120">
 <if_matched_sid>5760</if_matched_sid>
 <same_source_ip />
 <description>CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from same IP)</description>
 <mitre>
  <id>T1110.001</id>
 </mitre>
</rule>
```

2. Backed up the master config: sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak

```
[wazuh-user@wazuh-server ~]$ sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak
```

3. Edited the <active-response> block in ossec.conf to utilize the firewall-drop command. I set the timeout to 300 seconds (5 minutes) for safe testing.

```
[wazuh-user@wazuh-server ~]$ sudo nano /var/ossec/etc/ossec.conf
```

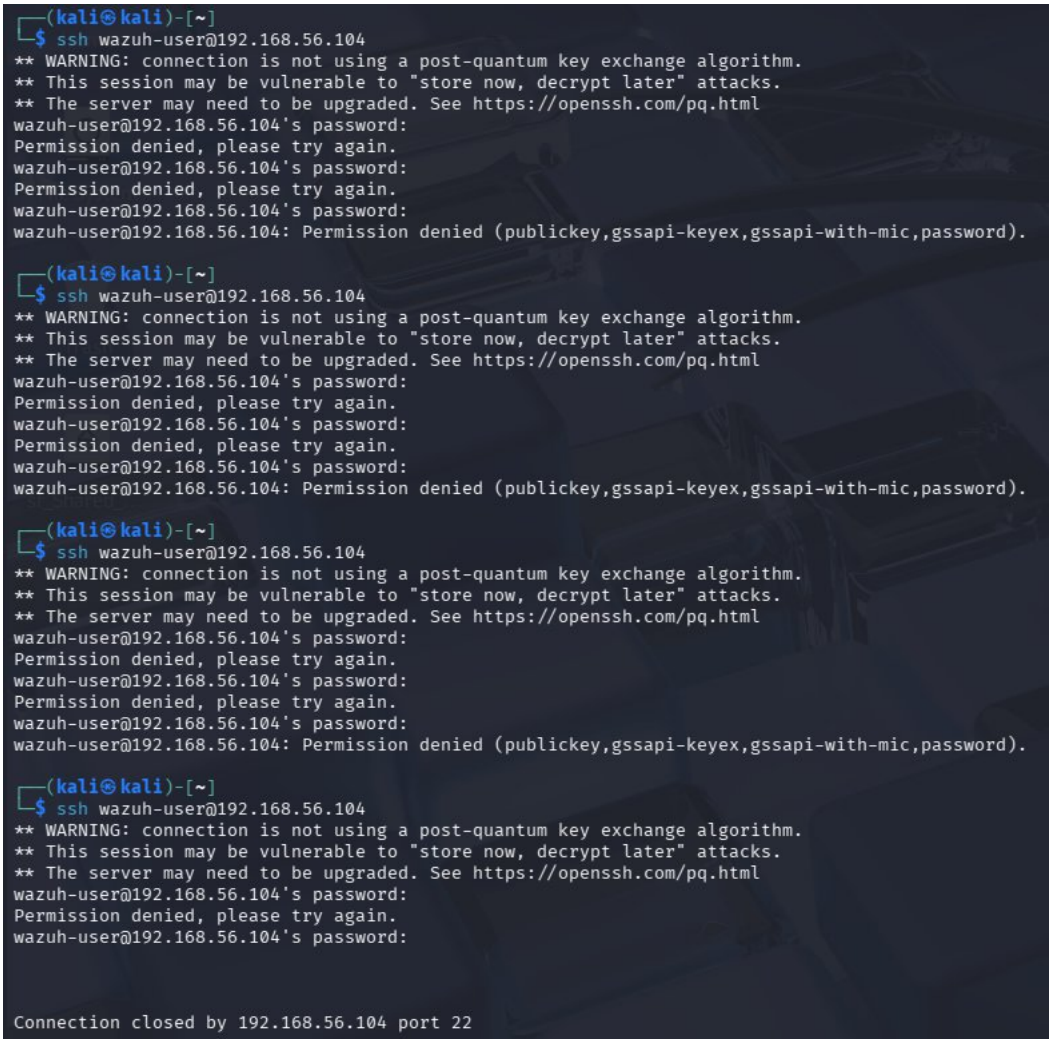
```
<!-- Active Responses -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100011</rules_id>
  <timeout>300</timeout>
</active-response>
```

4. Verified iptables was installed on the Amazon Linux server to ensure the script could execute properly.

```
[wazuh-user@wazuh-server ~]$ sudo yum install iptables -y
Last metadata expiration check: 11:12:48 ago on Sat Mar 14 20:08:11 2026.
Package iptables-nft-1.8.8-3.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
```

Attack and Verification

I initiated an SSH brute-force attack from Kali. On the 6th attempt, the terminal froze, and the connection was completely refused.



Running sudo iptables -L -n on the server proved that the Wazuh agent had successfully rewritten the host firewall, actively dropping all traffic from the attacker's IP (192.168.56.107).

```
[wazuh-user@wazuh-server ~]$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.56.107       0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.56.107       0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

The dashboard confirmed the entire kill chain: Login failures grouped into a brute-force alert, instantly followed by Rule 651 indicating the Active Response execution.

Mar 15, 2026 @ 12:24:33.2...	wazuh-server	PAM: Login session closed.	3	5502
Mar 15, 2026 @ 12:24:33.2...	wazuh-server	PAM: Login session opened.	3	5501
Mar 15, 2026 @ 12:24:33.2...	wazuh-server	Successful sudo to ROOT executed.	3	5402
Mar 15, 2026 @ 12:24:01.3...	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Mar 15, 2026 @ 12:24:01.2...	wazuh-server	CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from sa...	12	100011
Mar 15, 2026 @ 12:24:01.2...	wazuh-server	sshd: authentication failed.	5	5760
Mar 15, 2026 @ 12:23:59.2...	wazuh-server	sshd: authentication failed.	5	5760
Mar 15, 2026 @ 12:23:59.2...	wazuh-server	sshd: authentication failed.	5	5760
Mar 15, 2026 @ 12:23:57.2...	wazuh-server	sshd: authentication failed.	5	5760
Mar 15, 2026 @ 12:23:57.2...	wazuh-server	sshd: authentication failed.	5	5760

Risks and Safeguards of Active Response

While Active Response is powerful, deploying it in a production environment carries significant operational risk.

The primary risk is a **Self-Denial of Service (DoS)**. If a legitimate administrator forgets their password and attempts to log in multiple times, an overly aggressive Active Response configuration will instantly lock them out of the server. Furthermore, sophisticated attackers can leverage **IP Spoofing**. By forging their source IP to match a critical internal asset (such as the company's Domain Controller or primary DNS server), an attacker can intentionally trigger the brute-force rule, causing the SIEM to automatically ban the company's own infrastructure and taking the network offline.

Safeguards: Before enabling this in production, strict safeguards must be established. First, a robust <white_list> must be configured in ossec.conf containing all mission-critical IP addresses (Domain Controllers, vulnerability scanners, admin workstations) to ensure they can never be banned. Second, permanent bans should be avoided; using a timeout threshold (e.g., 5 to 15 minutes) ensures that if an accidental block occurs, the system will automatically self-heal and restore access without requiring emergency manual intervention.

End of Week 2 Report